

P3501R0: The ad-dressing of cats

Audience: EWG, CWG, LWG
S. Davis Herring <herring@lanl.gov>
Los Alamos National Laboratory
January 10, 2025

Introduction

The word “address” is used with several different meanings in the standard, each given merely implicit definitions. [intro.memory]/1 tells us “Every byte has a unique address.”, and pointers are said to “represent the address” of bytes of memory ([basic.compound]/3); [expr.delete]/5 and [class.mem.general]/22 (and some places in the library) refer to an order on such addresses, and [basic.align]/1 even talks about a number of bytes between addresses. There is also some sort of order invoked for bytes in [basic.compound]/3 and [intro.object]/8. However, no further use of those order and/or arithmetic structures is made: in particular, the integers resulting from `reinterpret_cast` are not described as addresses, except in the note in [expr.reinterpret.cast]/4 about the conversion being “unsurprising”.

Furthermore, the word is sometimes used to just refer to any use of the unary `&` operator (e.g., [basic.lookup.argdep]/3 and [dcl.ptr]/4) or to any pointer (e.g., [expr.delete]/11, [atomics.ref.pointer]/6, and [util.smartptr.atomic.shared]/2). Finally, sometimes it refers to the identity of an entity (e.g., [dcl.inline]/6, [temp.constr.order]/2.2, or [temp.explicit]/12).

There is also one significant defect where the word should be used but is not: [defns.order.ptr] refers to a total order over pointer values that `std::less` and company cannot possibly produce with real implementations. In particular, `std::less<void*>` cannot distinguish the pointer values of a pointer to an array and a pointer to its first element (which are not even pointer-interconvertible). Neither can `std::less<int*>` distinguish a pointer past one `int` (`&x+1`) with a pointer to another `int` (`&y`) that happens to be stored immediately after it.

This paper clarifies that addresses are totally ordered opaque labels for bytes of memory, rewriting [expr.eq] to use that order (as does [comparisons]) and [expr.rel] to use the appropriate subset of that order. ([expr.rel] results must continue to be otherwise unspecified because it is desirable that the result of `<` be able to differ between its usage directly and via `std::less`, whose total order must be unavailable during constant evaluation even if `<` provides it at runtime.) It changes the usages with other meanings, continuing in the [direction](#) already established there. It also clarifies the relationships between addresses, bytes, and objects.

Proposal

Associate an address with every pointer value, even pointers to functions and null pointers. Establish a total order on these addresses, retaining the possibility that objects are not contiguous in it to support either interleaving or concatenating segments in the `std::less` order. Define equality operators on pointers and all [comparisons] function objects in terms of this order. Introduce the term “ordered” for pointers where `<` is available during constant evaluation.

Do not alter the general implementation-defined support for any operation on pointers not valid in the context of that operation; note that such an alteration has been [separately proposed](#). Nor alter the description of `void*` values used with `operator new` and company strictly in terms of their addresses (and suitable created objects).

Additionally, correct the separate defective claim in [comparisons.general] that `std::less<void>` produces an ordered result for distinct arguments even though, if they have different types, the conversion to their composite pointer type ([`expr.rel`]/3) can cause them to become equal.

Use strictly “take the address” for the operation of `&`, but formally define “address of an overload set” (whose evaluation is deferred until after overload resolution). Avoid using “address of *E*” when “pointer to *E*” or “*E*” is correct.

Consequences

The only change of semantics (beyond the alignment of the library facilities with real implementations) is that comparisons involving one-past pointers are no longer unspecified (but still cannot be used in constant expressions). These changes to [`expr.eq`] merge cleanly with those from [P2434R2](#).

Wording

Relative to N5001.

#[defns.order.ptr]

Remove subclause. Forward its stable name to [intro.memory].

#[basic]

#[basic.memobj]

#[intro.memory]

Change paragraph 1:

- [...] The memory available to a C++ program consists of one or more sequences of contiguous bytes. Every byte has a unique address, which is a label drawn from a larger set equipped with an implementation-defined total order.

[Note: The order is independent of the sequence order of the bytes except as specified ([intro.object], [basic.stc.dynamic.allocation]). — end note]

#[intro.object]

Change paragraph 8:

- [...] Unless it is a bit-field ([class.bit]), an object with nonzero size shall occupy one or more bytes of storage, including every byte that is occupied in full or in part by any of its subobjects. An object of trivially copyable or standard-layout type ([basic.types.general]) shall occupy contiguous bytes of storage. The order of the addresses of the bytes occupied by any object is consistent with that of the bytes ([intro.memory]).

[Note: The address of a byte occupied by one complete object can intervene between those of the bytes occupied by another complete object. — end note]

#[basic.align]

Change paragraph 1:

- Object types have *alignment requirements* ([basic.fundamental,basic.compound]) which place restrictions on the addresses at which an object of that type may be allocated. An *alignment* is an implementation-defined integer value representing the number spacing of successive bytes between successive at whose addresses at which a given object can be allocated. [...]

#[basic.life]

Change paragraph 7:

- Before the lifetime of an object has started but after the storage which the object will occupy has been allocated^[...] or, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, any a pointer that represents the address of the storage location where to the object will be or was located may be used but only in limited ways. [...]

Change paragraph 9:

- [...]
- [Note: If these conditions are not met, a pointer to the new object can be obtained from a pointer that represents the with its address of its storage by calling `std::launder` ([ptr.launder]). — end note]

#[basic.stc.dynamic.allocation]

Insert before paragraph 2:

- The address of a block of storage is the address of the first byte in it. The addresses of the bytes in any block of storage are in increasing order ([intro.memory]).

Change paragraph 2:

- An allocation function attempts to allocate the requested amount of storage. If it is successful, it returns the address of ~~the start of~~ a block of storage whose length in bytes is at least as large as the requested size. The order, contiguity, and initial value of storage allocated by successive calls to an allocation function are unspecified. [...] Furthermore, for the library allocation functions in [new.delete.single] and [new.delete.array], the address of p0 ~~represents the address is that~~ of a block of storage disjoint from the storage for any other object accessible to the caller.

Change paragraph 3:

- For an allocation function other than a reserved placement allocation function ([new.delete.placement]), the pointer returned on a successful call shall ~~represent the~~ have as its address that of storage that is aligned as follows:

#[basic.compound]

Change and split paragraph 3:

- [...]what can be done with them ([basic.types.general]).
- Every value of pointer type holds an address ([intro.memory]) that is not in general unique. A value of a pointer type that is a pointer to or past the end of an object ~~represents the address~~ holds the address of ~~the first byte in memory ([intro.memory]) occupied by~~ the object [*Footnote: For an object that is not within its lifetime, this is the address of the first byte in memory that it will occupy or used to occupy. — end footnote*] or of the first byte in memory after the end of the storage occupied by the object, respectively.

[*Note: A pointer past the end of an object ([expr.add]) is not considered to point to an unrelated object of the object's type, even if the unrelated object is located at that address. — end note*]

Every pointer value that points to a function holds a unique address. All null pointer values hold a single unique address. Every invalid pointer value holds a unique address. These addresses are not associated with any byte.
- For purposes of pointer arithmetic ([expr.add]) ~~and comparison ([expr.rel,expr.eq])~~, a pointer past the end of the last element of an array x of n elements [...]

#[expr]

#[expr.compound]

#[expr.static.cast]

Change paragraph 13:

- [...] If the original pointer value ~~represents the~~ points to or past an object and holds an address ~~A of a byte in memory and A that~~ does not satisfy the alignment requirement of T, then the resulting pointer value ([basic.compound]) is unspecified. [...]

#[expr.unary]

#[expr.unary.op]

Change paragraph 3:

- The operand of the unary & operator shall be an lvalue of some type T. The unary-expression is said to take the address of its operand.
[...]

Change paragraph 6:

- An address of an overload set is a unary-expression that takes the address of an id-expression whose terminal name refers to an overload set. [Note: The address of an overload set ~~({over})~~ can be taken only in a context that uniquely determines which function is referred to (see [over.match.call.general], [over.over]). Since the context can affect whether the operand is a static or non-static member function, the context can also affect whether the expression has type “pointer to function” or “pointer to member function”. — end note]

#[expr.new]

Change paragraph 17:

- [...] For arrays of char, unsigned char, and std::byte, the difference between the result of the *new-expression* and the ~~address~~ value returned by the allocation function (converted to the same type) shall be an integral multiple of the strictest fundamental alignment requirement ([basic.align]) of any object type whose size is no greater than the size of the array being created.

#[expr.delete]

Change paragraph 10:

- [...] When a *delete-expression* is executed, the selected deallocation function shall be called with ~~the address of~~ a pointer to the deleted object in a single-object delete expression, or ~~the address of the deleted object~~ such a pointer suitably adjusted for

the array allocation overhead ([*expr.new*]) in an array delete expression, as its first argument.

[*Note*: [...] — *end note*]

#[*expr.spaceship*]

Change paragraph 6:

- [...]

In this case, *p* <=> *q* is of type `std::strong_ordering` and the result is defined by the following rules:

1. If ~~two pointer operands~~ *p* and *q* compare equal ([*expr.eq*]), *p* <=> *q* yields `std::strong_ordering::equal`;
2. otherwise, if *p* and *q* ~~compare unequal~~ are ordered ([*expr.rel*]), *p* <=> *q* yields `std::strong_ordering::less` if ~~*q* compares greater than *p*~~ *p* < *q* is true and `std::strong_ordering::greater` ~~if *p* compares greater than *q*~~ if *p* > *q* (*expr.rel*) otherwise;
3. otherwise, the result is unspecified.

#[*expr.rel*]

Replace paragraphs 4–6:

- The result of comparing unequal pointers to objects^[...] is defined in terms of a partial order consistent with the following rules:

[...]

- If two operands *p* and *q* compare equal ([*expr.eq*]), [...]
- If both operands (after conversions) are of arithmetic or enumeration type, [...]

- *p* > *q* yields *q* < *p*, *p* <= *q* yields !(*p* > *q*), and *p* >= *q* yields !(*p* < *q*).

- If their operands *p* and *q* are pointers:
1. *p* and *q* are *ordered* if an object *O* exists such that *p* and *q* each point to or past an object that is or is nested within *O*. In that case, *p* < *q* yields `true` if the address held by *p* precedes that held by *q* and `false` otherwise.
 2. Otherwise, if *p* and *q* hold the same address, *p* < *q* yields `false`.
 3. Otherwise, the result of *p* < *q* is unspecified.
- Otherwise, *p* < *q* yields `true` if the (converted) value of *p* is less than that of *q* and `false` otherwise.

#[*expr.eq*]

Change paragraph 3:

- If at least one of the converted operands is a pointer, pointer conversions ([conv.ptr]), function pointer conversions ([conv.fctptr]), and qualification conversions ([conv.qual]) are performed on both operands to bring them to their composite pointer type ([expr.type]). If the pointer values hold the same address, they compare equal; otherwise Comparing pointers is defined as follows:
 1. ~~If one pointer represents the address of a complete object, and another pointer represents the address one past the last element of a different complete object, [...] the result of the comparison is unspecified.~~
 2. ~~Otherwise, if the pointers are both null, both point to the same function, or both represent the same address ([basic.compound]), they compare equal.~~
 3. ~~Otherwise,~~ the pointers they compare unequal.

#[expr.const]

Change and split bullet (10.25):

- a three-way comparison ([expr.spaceship]); or relational ([expr.rel]), or operator whose (pointer) operands are unordered;
- an equality ([expr.eq]) operator whose (pointer) operands hold the address of one complete object and the address held by a pointer past another complete object; ~~where the result is unspecified~~

#[dcl.dcl]

#[dcl.inline]

Change paragraph 6:

- [Note: An inline function or variable with external or module linkage can be defined in multiple translation units ([basic.def.odr]), but is ~~one~~ a single entity ~~with one address~~. A type or static variable defined in the body of such a function is therefore a single entity. — end note]

#[dcl.array]

Change paragraph 6:

- An object of type “array of N U” consists of a ~~contiguously allocated~~ non-empty ~~set~~ sequence of N subobjects of type U, known as the *elements* of the array, and numbered 0 to N-1. The first element has the same address as the array; each other element occupies the bytes immediately following the previous.
- [Note: Because the array is an object, the addresses of the elements are in the same order as the elements. — end note]

#[dcl.fct.def.coroutine]

Change paragraph 12:

- [...] The selected deallocation function shall be called with the address of the block of storage to be reclaimed value returned by the allocation function as its first argument. If a deallocation function with a parameter of type `std::size_t` is used, the size of the block is passed as the corresponding argument.

#[class.mem.general]

Change paragraph 22:

- ~~[Note: Among N~~ non-variant non-static data members of non-zero size ([intro.object]) ~~are allocated so that,~~ later members ~~have higher addresses~~ occupy later bytes within a class object ~~((expr.rel))~~.

[Note: Implementation alignment requirements can cause two adjacent members not to be allocated immediately after each other; so can requirements for space for managing virtual functions ([class.virtual]) and virtual base classes ([class.mi]).
— end note]

#[temp]

#[temp.param]

Change the example in paragraph 8:

```
&i; // error: taking the address of a non-const lvalue
```

#[temp.constr.order]

Change bullet (2.2):

- the address of a non-template function selected from an overload set ([over.over]),

#[temp.func.order]

Change paragraph 1:

- If multiple function templates share a name, the use of that name can be ambiguous because template argument deduction ([temp.deduct]) may identify a specialization for more than one function template. *Partial ordering* of overloaded function template declarations is used in the following contexts to select the function template to which a function template specialization refers:
 1. during overload resolution for (a call to) a function template specialization ([over.match.best], [over.over]);
 2. when the address of a function template specialization is taken;

3. when a placement operator delete that is a function template specialization is selected to match a placement operator
new ([basic.stc.dynamic.deallocation,expr.new]);
4. when a friend function declaration ([temp.friend]), an explicit instantiation ([temp.explicit]) or an explicit specialization ([temp.expl.spec]) refers to a function template specialization.

#[temp.deduct]

Deducing template arguments ~~taking the address of~~ for a named function template # [temp.deduct.funcaddr]

Change the section title.

Change paragraph 1:

- Template arguments can be deduced from the type specified when ~~taking the address of~~ resolving an overload set ([over.over]). [...]

#[temp.deduct.type]

Change paragraph 10:

- [...] If P and A are function types that originated from deduction when ~~taking the address of~~ naming a function template ([temp.deduct.funcaddr]) or when deducing template arguments from a function declaration ([temp.deduct.decl]) and P_i and $A_{i_}$ are parameters of the top-level parameter-type-list of P and A, respectively, [...]

#[library]

#[byte.strings]

Change paragraph 1:

- A *null-terminated byte string*, or NTBS, is a character sequence whose ~~highest-addressed~~ last element with defined content has the value zero (the *terminating null character*); no other element in the sequence has the value zero. [...]

#[res.on.arguments]

Change bullet (1.2):

- If a function argument is described as being an array, the pointer actually passed to the function shall have a value such that all ~~address~~ pointer computations and accesses to objects (that would be valid if the pointer did point to the first element of such an array) are in fact valid.

#[support]

#[support.dynamic]

#[new.delete]

#[new.delete.single]

Change paragraphs 10 and 18 identically:

- *Preconditions:* `ptr` is a null pointer or its value represents it holds the address of a block of memory allocated by an earlier call to a (possibly replaced) operator `new(std::size_t)` or operator `new(std::size_t, std::align_val_t)` which has not been invalidated by an intervening call to operator `delete`.

#[new.delete.array]

Change paragraphs 9 and 15 identically:

- *Preconditions:* `ptr` is a null pointer or its value represents it holds the address of a block of memory allocated by an earlier call to a (possibly replaced) operator `new[](std::size_t)` or operator `new[](std::size_t, std::align_val_t)` which has not been invalidated by an intervening call to operator `delete[]`.

#[ptr.laundry]

Change paragraph 2:

- *Preconditions:* `p` represents the address `A` of a byte in memory. An object `X` that is within its lifetime ([basic.life]) and whose type is similar ([conv.qual]) to `T` is located at has the address `A` held by `p`. All bytes of storage that would be reachable through ([basic.compound]) the result are reachable through `p`.

#[coroutine.handle.general]

Change paragraph 1:

- An object of type `coroutine_handle<T>` is called a *coroutine handle* and can be used to refer to a suspended or executing coroutine. A `coroutine_handle` object whose member `address()` returns a null pointer value does not refer to any coroutine. Two `coroutine_handle` objects refer to the same coroutine if and only if their member `address()` returns the same non-null values that compare equal.

#[syserr.errcat.nonvirtuals]

Change paragraph 2:

- *Returns:* `compare_three_way()(this, &rhs)`.

[Note: compare_three_way ([comparisons.three.way]) provides a total ordering for pointers addresses. — end note]

#[mem]

#[memory]

#[ptr.align]

Change bullet (1.2):

- | ptr represents holds the address of contiguous storage of at least space bytes

Change paragraph 2:

- | *Effects:* If it is possible to fit size bytes of storage aligned by alignment into the buffer pointed to by ptr with length space, the function updates assigns to ptr to represent a pointer that holds the first possible address of such storage and decreases space by the number of bytes used for alignment. Otherwise, the function does nothing.

#[specialized.addressof]

Change paragraph 1:

- | *Returns:* The actual address of A pointer to the object or function referenced by r, even in the presence of an overloaded operator&.

#[util.smartptr.shared.create]

Change paragraph 5:

- | *Returns:* A shared_ptr instance that stores and owns the address of a pointer to the newly constructed object.

Change paragraph 10:

- | *Remarks:* The shared_ptr constructors called by these functions enable shared_from_this with the address of a pointer to the newly constructed object of type T.

#[mem.res]

#[mem.poly allocator.mem]

Change paragraph 15:

- | *Effects:* Construct a T object in the storage whose address is represented by that held by p by uses-allocator construction with allocator *this and constructor arguments std::forward<Args>(args)....

#[mem.res.pool.ctor]

Change paragraph 1:

- | *Preconditions:* upstream ~~is the address of~~ points to a valid memory resource.

#[mem.res.monotonic.buffer.ctor]

Change paragraph 1:

- | *Preconditions:* upstream ~~is the address of~~ points to a valid memory resource.
initial_size, if specified, is greater than zero.

Change paragraph 3:

- | *Preconditions:* upstream ~~is the address of~~ points to a valid memory resource.
buffer_size is no larger than the number of bytes in buffer.

#[utilities]

#[comparisons]

#[comparisons.general]

Change paragraph 2:

- | For templates less, greater, less_equal, and greater_equal, the specializations for any pointer type yield a result ~~consistent with~~ based on the implementation-defined strict total order over pointers addresses (~~[defns.order.ptr~~ intro.memory]).

[Note: If ~~a < b~~ is well-defined for pointers a and b of type P ~~are~~ ordered ([expr.rel]), then (a < b) == less<P>()(a, b), (a > b) == greater<P>()(a, b), and so forth. — end note]

For template specializations less<void>, greater<void>, less_equal<void>, and greater_equal<void>, if the call operator calls a built-in operator comparing pointers, the call operator yields a result ~~consistent with~~ based on the implementation-defined strict total order over ~~pointers~~ the addresses held by the arguments after conversion to their composite pointer type.

#[comparisons.three.way]

Change bullet (3.1):

- | If the expression std::forward<T>(t) <=> std::forward<U>(u) results in a call to a built-in operator <=> comparing pointers ~~of type P~~, returns strong_ordering::less if (the converted value of) t precedes u in the implementation-defined strict total order over pointers addresses (~~[defns.order.ptr~~ intro.memory]), strong_ordering::greater if u precedes t, and otherwise strong_ordering::equal.

#[range.cmp]

Change paragraph 1:

- | Constraints: T and U satisfy `model` equality_comparable_with.

Remove paragraph 2:

- | *Preconditions:* If the expression `std::forward<T>(t) == std::forward<U>(u)` results in a call to a built-in operator `==` comparing pointers of type P, the conversion sequences from both T and U to P are equality-preserving ([concepts.equality]); otherwise, T and U model equality_comparable_with.

Change paragraph 3:

- | *Effects:*
 1. ~~If the expression `std::forward<T>(t) == std::forward<U>(u)` results in a call to a built-in operator `==` comparing pointers: returns false if either (the converted value of) `t` precedes `u` or `u` precedes `t` in the implementation-defined strict total order over pointers ([defns.order.ptr]) and otherwise true.~~
 2. Otherwise, `E`quivalent to: return `std::forward<T>(t) == std::forward<U>(u)`;

Change bullet (10.1):

- | If the expression `std::forward<T>(t) < std::forward<U>(u)` results in a call to a built-in operator `<` comparing pointers: returns true if (the converted value of) `t` precedes `u` in the implementation-defined strict total order over pointers `addresses` ([defns.order.ptr `intro.memory`]) and otherwise false.

#[func.wrap.ref.ctor]

Change paragraph 4:

- | *Effects:* Initializes *bound-entity* with *f*, and *thunk_ptr* with `the address of a pointer to` a function *thunk* such that *thunk*(*bound-entity*, *call-args...*) is expression-equivalent ([defns.expression.equivalent]) to `invoke_r<R>(f, call-args...)`.

Change paragraph 7:

- | *Effects:* Initializes *bound-entity* with `addressof(f)`, and *thunk_ptr* with `the address of a pointer to` a function *thunk* such that *thunk*(*bound-entity*, *call-args...*) is expression-equivalent ([defns.expression.equivalent]) to `invoke_r<R>(static_cast<cv T&>(f), call-args...)`.

Change paragraph 11:

- | *Effects:* Initializes *bound-entity* with a pointer to an unspecified object or null pointer value, and *thunk_ptr* with `the address of a pointer to` a function *thunk* such that *thunk*(*bound-entity*,

`call-args...`) is expression-equivalent ([defns.expression.equivalent]) to `invoke_r<R>(f, call-args...)`.

Change paragraph 15:

- *Effects:* Initializes *bound-entity* with `addressof(obj)`, and *thunk-ptr* with ~~the address of~~ `a pointer to` a function *thunk* such that `thunk(bound-entity, call-args...)` is expression-equivalent ([defns.expression.equivalent]) to `invoke_r<R>(f, static_cast<cv T&>(obj), call-args...)`.

Change paragraph 20:

- *Effects:* Initializes *bound-entity* with `obj`, and *thunk-ptr* with ~~the address of~~ `a pointer to` a function *thunk* such that `thunk(bound-entity, call-args...)` is expression-equivalent ([defns.expression.equivalent]) to `invoke_r<R>(f, obj, call-args...)`.

#[container.alloc.reqmts]

Change bullet (2.2):

- [...] where `p` ~~is~~ `holds` the address of the uninitialized storage for the element allocated within `X`.

#[text]

#[locale.time.get.virtuals]

Change paragraph 15:

- *Remarks:* It is unspecified whether multiple calls to `do_get()` with ~~the address of~~ the same `tm` object will update the current contents of the object or simply overwrite its members. Portable programs should zero out the object before invoking the function.

#[re.tokiter]

#[re.tokiter.cnstr]

Change paragraph 4:

- Each constructor then sets `N` to 0, and `position` to `position_iterator(a, b, re, m)`. If `position` is not an end-of-sequence iterator the constructor sets `result` to ~~the address~~ `a pointer to the beginning` of the current match. Otherwise if any of the values stored in `subs` is equal to `-1` the constructor sets `*this` to a suffix iterator that points to the range `[a,b)`, otherwise the constructor sets `*this` to an end-of-sequence iterator.

#[re.tokiter.incr]

Change paragraphs 3 and 4:

- Otherwise, if `N + 1 < subs.size()`, increments `N` and sets `result` to ~~the address~~ a pointer to the beginning of the current match.
- Otherwise, sets `N` to 0 and increments `position`. If `position` is not an end-of-sequence iterator the operator sets `result` to ~~the address~~ a pointer to the beginning of the current match.

#[class.gslicing.overview]

Change paragraph 4:

- It is possible to have degenerate generalized slices in which an ~~address~~ element is repeated.

#[streambuf.reqts]

Change paragraph 2:

- [...] The three pointers are:
 1. the *beginning pointer*, ~~or lowest~~ to the first element ~~address~~ in the array (called `xbeg` here);
 2. the *next pointer*, ~~or~~ to the next element ~~address~~ that is a current candidate for reading or writing (called `xnext` here);
 3. the *end pointer*, ~~or first element address beyond~~ to the end of the array (called `xend` here).

#[atomics]

[*Drafting note*: While there is of course no normative wording on the subject of memory mapping or multiple processes, the use of “address” in `[atomics.lockfree]/5` is otherwise consistent with the definitions here. — *end drafting note*]

#[atomics.ref.pointer]

Change paragraph 9:

- *Remarks*: The result may be an ~~undefined address~~ invalid pointer value, but the operations otherwise have no undefined behavior.

#[atomics.types.generic]

#[atomics.types.operations]

Change paragraph 3:

- *Effects:* Initializes the object with the value desired. Initialization is not an atomic operation ([intro.multithread]). [*Note:* It is possible to have an access to an atomic object A race with its construction, for example by communicating the address of a pointer to the just-constructed object A to another thread via `memory_order::relaxed` operations on a suitable atomic pointer variable, and then immediately accessing A in the receiving thread. This results in undefined behavior. — *end note*]

#[atomics.types.pointer]

Change paragraph 8:

- *Remarks:* The result may be an undefined address invalid pointer value, but the operations otherwise have no undefined behavior.

#[util.smartptr.atomic]

#[util.smartptr.atomic.shared]

Change paragraph 2:

- *Effects:* Initializes the object with the value desired. Initialization is not an atomic operation ([intro.multithread]). [*Note:* It is possible to have an access to an atomic object A race with its construction, for example, by communicating the address of a pointer to the just-constructed object A to another thread via `memory_order::relaxed` operations on a suitable atomic pointer variable, and then immediately accessing A in the receiving thread. This results in undefined behavior. — *end note*]

#[util.smartptr.atomic.weak]

Change paragraph 2:

- *Effects:* Initializes the object with the value desired. Initialization is not an atomic operation ([intro.multithread]). [*Note:* It is possible to have an access to an atomic object A race with its construction, for example, by communicating the address of a pointer to the just-constructed object A to another thread via `memory_order::relaxed` operations on a suitable atomic pointer variable, and then immediately accessing A in the receiving thread. This results in undefined behavior. — *end note*]

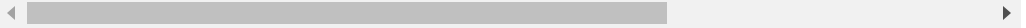
#[diff]

#[diff.cpp23.expr]

Change paragraph 2:

- **Affected subclauses:** [expr.rel] and [expr.eq]
Change: Comparing two objects of array type is no longer valid.
Rationale: The old behavior was confusing since it compared not the contents of the two arrays, but **pointers to** their **addresses** **first elements**.
Effect on original feature: A valid C++2023 program directly comparing two array objects is rejected as ill-formed in this document. [Example:

```
int arr1[5];  
int arr2[5];  
bool same = arr1 == arr2;           // ill-formed; previous  
bool idem = arr1 == +arr2;          // compare addresses p  
bool less = arr1 < +arr2;           // compare addresses p
```


— end example]

#[diff.expr]

Change paragraph 5:

- **Affected subclauses:** [expr.rel] and [expr.eq]
Change: C allows directly comparing two objects of array type; C++ does not.
Rationale: The behavior is confusing because it compares not the contents of the two arrays, but **pointers to** their **addresses** **first elements**.
Effect on original feature: Deletion of semantically well-defined feature that had unspecified behavior in common use cases.
[...]